

Lab Report

Summer 2026

Soeun Jang
Virginia Tech
sophiaj04@vt.edu

June 12, 2026

Contents

Introduction	3
Low-Rank SVD	3
Step-and-Truncate	3
Taylor series	4
Strategy A vs Strategy B	4
Sensitivity matrix	4
Cheap gradient	5
Tensor Operations	5
Conclusion	5

Introduction

This report summarizes key concepts from Lab 1–3: low-rank SVD, Taylor series time stepping, Step-and-Truncate methods, Tucker tensors, and low-rank PDE solvers. Each lab focuses on the methods to express high dimensional data effectively and compute the data.

- High dimensional PDEs are too large to compute directly.
- Therefore, we compress them using low-rank structures such as SVD or Tucker.
- These compressed forms allow efficient evolution of PDE solutions.

The workflow across all labs follows the same pattern:

1. Compress the object (matrix or tensor).
2. Apply the PDE operator in low-rank form.
3. Truncate to prevent rank explosion.
4. Use Taylor or RK4 for time stepping.
5. Analyze how the rank changes over time.
6. Use low-rank structure for gradient computation or extend to 3D with Tucker.

The goal of these labs is to compute high-dimensional PDEs fast while keeping computational cost low. Real PDE solutions mainly contain low-rank structure, meaning they can be represented accurately using compressed forms such as SVD or Tucker.

Low-Rank SVD

This is the matrices in low rank SVD form and helps to learn how to truncate them efficiently without computing full SVD. Low-Rank helps to reduce computational cost ($O(nr)$ becomes $O(n^2)$) which means data is focused on a few important singular values. This also saves the storage and deletes the noise.

Pros:

- Reduces computational cost.
- Saves storage.
- Removes noise by focusing on dominant singular values.

Cons:

- Full SVD is expensive.
- Hard to choose the correct rank.

Step-and-Truncate

This is the method that while processing time stepping method by Taylor series, it helps to prevent the rank getting bigger with the truncation every steps. Since it keeps the rank, its computational cost is consistent so it can save the memories, which means we can simulate for long term.

Pros:

- Prevents rank explosion.
- Keeps computational cost consistent.

- Enables long-term simulation.

Cons:

- Truncation introduces error.
- Frequent truncation increases cost.

Taylor series

The n-th order Taylor expansion improves the PDE solution. It does contain high accuracy but its computational cost is too expensive. This is accurate method and simple to work, but repeated calculation of truncation makes the speed slow (especially in Tucker structure).

Pros:

- High accuracy.
- Simple to implement.

Cons:

- Computationally expensive.
- Repeated truncation slows performance.

Strategy A vs Strategy B

A represents truncating each term and B represents truncating only once at the end. A is more accurate to keep rank low but B is faster to compute since it does not have to compute until the end.

Pros of Strategy A:

- More stable.
- Keeps rank low.

Cons of Strategy A:

- Slower due to frequent truncation.

Pros of Strategy B:

- Faster computation.

Cons of Strategy B:

- Risk of temporary rank growth.

Sensitivity matrix

Computing sensitivity matrix indicates the way how the PDE solution changes by parameters. Since the sensitivity matrix is big, it is necessary to contain low rank structure. However, low-rank structure is needed and Dense structure only stores $O(n^2)$, cannot compute but it can compute its gradient.

Pros:

- Shows how solution changes with parameters.
- Low-rank structure makes computation feasible.

Cons:

- Dense sensitivity matrix is too large to store.

Cheap gradient

Computing gradient using only low-rank factors without full SVD matrices, only with Tucker and SVD factors to reduce the computational cost but gradient can be inaccurate when rank is too small and truncational error might affect the gradient.

Pros:

- Fast and memory-efficient.
- Works well in optimization loops.

Cons:

- Inaccurate when rank is too small.
- Truncation error may affect gradient.

Tensor Operations

Since 3D tensor contains too big size, we save this tensor as small core and basis by compressing efficiently. It saves space to store and computational costs, also we can observe its mode structures. However, it is complicated. In step-and-truncate in Tucker, we keep low-rank in compressed form since dense tensor is too big when we solve 3D PDE. This costs a lot by truncating especially in Taylor method but it keeps rank stable and accuracy with reducing costs for 3D PDE.

Pros:

- Saves storage.
- Reduces computational cost.
- Reveals mode structure.

Cons:

- Tucker operations are complex.
- Truncation is expensive.

Conclusion

Lab 1-3 allowed us to see directly why the low-rank method is important and how effective it is in real PDE calculations. Beginning with the matrix SVD, solving 2D PDE and expanding to 3D PDE with Tucker tensor. Based on the results, many PDE solutions and sensitivity matrices actually contain low-rank structure, so it was impossible to calculate accurately enough with only a small rank without computing the full SVD and tensors. Additionally, it keeps the rank low with the proper use of truncation and maintain the accuracy. In conclusion, low-rank SVD represents the tool as solving high dimensional problem fast and efficient.